

Love the Problem, Not Your Solution

Problems, not solutions, create space for innovation.



ASH MAURYA

PROBLEM/SOLUTION FIT

I recently got asked about the most common pitfall that trips up entrepreneurs. Top on my list is: **Falling in love with your solution.** I've previously labeled this pre-disposition for the solution as the "Innovator's Bias."

In science, one of the ways we attempt to overcome this kind of bias is through reasoning from first principles. The same can also be applied to business.

With first principles you boil things down to the most fundamental truths...

and then reason up from there.

– Elon Musk

As you'll see in this post, the "Innovator Bias" is a sneaky troll—rearing its ugly head, not just during ideation but throughout the innovation lifecycle, often when you least expect it. At each step, some of the most fundamental truths come from a deep understanding of problems before solutions.

The Big Idea

When we first get hit by an idea, the solution is what we most clearly see and what we spend most of our energy towards. But most products fail—not because we fail to build out our solution, but because we fail to solve a "big enough" customer problem.

All your initial energy should be channeled towards finding evidence of a monetizable problem, not towards acquiring more resources to build out your solution.

The combination of a big enough monetizable problem and a unique solution = Problem/Solution Fit.

So how do you find a "big enough" problem?

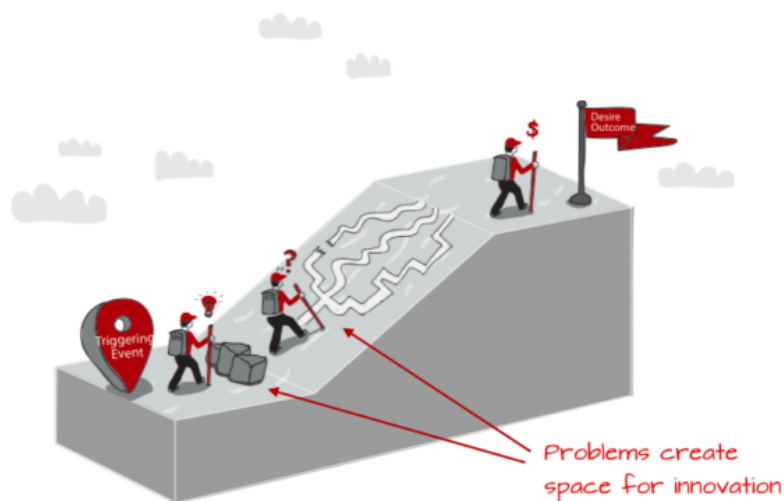
Start by recognizing that your true job is to create a customer (not your solution).

Customers are results or outcome driven.

Look for a job they are trying to get done

(jobs-to-be-done) and study how they are getting it done (existing alternatives).

If the job is adequately getting done, that's bad news for you because it's hard to displace an existing solution with a similar-sounding value proposition. If, on the other hand, you find the job isn't getting done "well enough," that's great news for you. The obstacles or problems getting in the way of the customer achieving their desired outcome are where you'll find space for innovation.



This emphasis on problem versus solution was the core mind shift I wanted to get across with the Lean Canvas and why I modified the original Business Model Canvas.

When I heard entrepreneurs pitch their Business Model Canvases, I heard a lot about what they were going to build (value proposition/solution) and a lot about how they were going to deliver it to customers (channels, key activities, key partners, key resources, customer relationships). But I

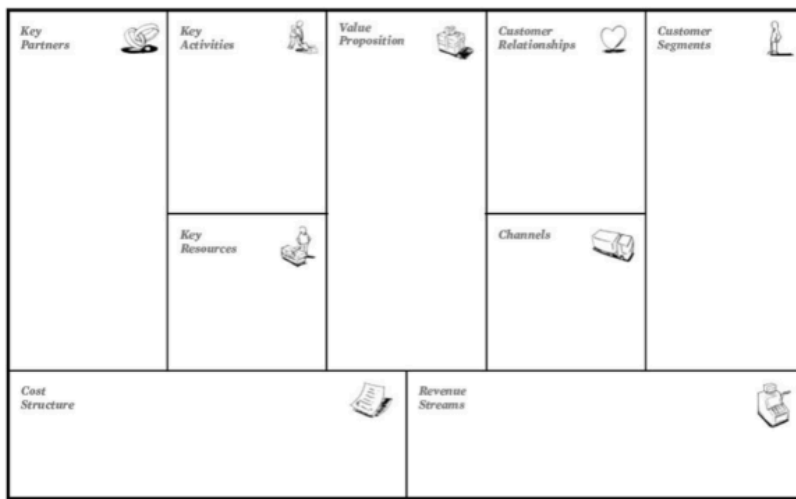
didn't hear anything about why customers would need or want your solution in the first place or how you would get them to switch from what they are doing today to your solution.

No problems in your business model is a problem.

The Lean Canvas isn't a better Business Model Canvas but a **different** canvas altogether. Even though I changed 4 boxes, the addition of the Problem box continues to generate the biggest mind shift in the resulting business model.



Lean Canvas
Customer Problem centric



Business Model Canvas

Solution Delivery centric

Pitfall: Most big ideas are too solution-centric.

Antidote: Love the Problem, Not Your Solution.

The Ever-growing Feature Backlog

Let's fast forward to a launched product with many customers. With lots of customers come lots of new feature requests. Who do you listen to?

If you listen to all your customers, pretty soon, you'll have a bloated monster on your hands. Even if you listen to your most valuable customers, you might still build stuff that they don't use. This is because most feature requests are framed as solutions, not problems. And customers are often not good at devising solutions—even to their problems.

"It's not the customer's job to know what they want."

-Steve Jobs

A better way to prioritize your customer feature requests is by first understanding the

root problem that triggered the request in the first place. Where were they? What were they trying to do? Why?

You can answer all these questions by simply applying the same jobs-to-be-done thinking process from above.

Here's an example from our LEANSTACK software:

Feature request 1: I'd like to be able to export my Lean Canvas as a PDF.

Feature request 2: I'd like to be able to change the colors on the Lean Canvas.

Feature request 3: I'd like to be able to change the fonts on the Lean Canvas.

Each of these was a simple enough feature request, but rather than just implementing them, we got the requestors on the phone. We asked them what they were trying to accomplish (outcome) and explored why the current product failed them (problem). In this case, we learned that these users wanted to use their Lean Canvases in investor presentations, and the default view was not visually interesting enough.

Once you understand the job, the axes of “better” get clearer.

Instead of implementing their feature requests verbatim, we mocked up a “presentation mode” feature and showed it to them. That's what we ended up building.

Pitfall: Most customer feature requests are framed as solutions, but customers are often not good solution designers.

Antidote: Love the Problem, Not Your Solution.

The Curse of Specialization

As you grow any product, things start to go downhill. To identify and prioritize issues, we invest in metrics. Good metrics help expose the leakiest buckets or bottlenecks in your business model. But simply asking your team for solutions to poor metrics can also cause an avalanche of internal feature requests in your backlog.

Here's another example from our LEANSTACK product.

Our metrics were starting to trend downward on completion rates of new Lean Canvases (activation). I brought this up in our weekly standup meeting, and here's what happened:

1. My designer proposed that we simplify the Lean Canvas creation workflow.
2. My developer proposed that we add more features to the workflow, and
3. My marketer wanted to implement lifecycle emails that guided users through the workflow.

Notice how the “Innovator’s Bias” sneaked in here. Each of us is trained to be good at certain disciplines, and when confronted with a problem, we automatically reach for our strongest tool. It's no surprise that my

Subscribe to the Newsletter

Join thousands of founders for battle-tested recipes, strategies, and how-tos for achieving product/market fit systematically.

Email Address

Send
Me
Weekly
Startup
Tips →

SHARE THIS ARTICLE ON:



designer came up with a design solution, my developer came up with a technical solution, and my marketer came up with a, yes, you guessed it, marketing solution.

Rather than falling into the trap of collectively loving our shiny new features after one too many overbuilt features, we have learned to always fall back to first principles. We look for evidence of the problem before formulating or proposing any solutions.

The problem with relying on metrics alone is that while they can tell you what's going wrong, they often don't tell you why. To get to why you usually have to devise a learning experiment. In this case, instead of automatically buying into a UX solution, we decided to run usability tests with new users, which invalidated UX as the reason for the drop in activation rates. We did uncover something else entirely that led to a different solution that worked.

Pitfall: Metrics can only tell you what's going wrong, not why, and your team will have a "different good idea" for how to fix it.

Antidote: Love the Problem, Not Your Solution.

Avoiding Failure

None of us like to fail, but when I think of avoiding failure, I'm often reminded of "The Hitchhiker's Guide to the Galaxy":

There is an art to flying, or rather a knack. Its knack lies in learning to

*throw yourself at the ground and miss.
... Clearly, it is this second part, the
missing, that presents the difficulties.
– Douglas Adams*

We spend so much time trying to “miss” failure that we fail to realize that: **Failing is a necessary pre-condition for a breakthrough.**

If you run an experiment and only validate what you already expected to happen, you can pat yourself on the back, but there was no breakthrough. It is only through the exploration of the unexpected that you achieve breakthroughs. The field of science and business alike is riddled with stories of such accidental breakthroughs: Penicillin, X-rays, Microwaves, Post-it Notes, Velcro, etc.

Breakthrough insights are often hidden inside failed experiments.

Failing is par for the course when we attempt new things. There is a reason why the hockey-stick curve is flat at the beginning. The trick to learning your way towards breakthrough versus avoiding failure is three-fold:

1. Eliminate big wipe-out types of failures by employing small, fast, additive experiments.
2. Remove failure from your vocabulary. I prefer the term “unexpected outcomes.”
3. Finally, chase every unexpected outcome to understand why.

A pivot not grounded in learning is a disguised “see what sticks” strategy.

Pitfall: *Running away from failure only delays true breakthroughs.*

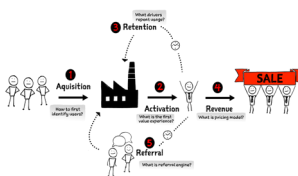
Antidote: *Love the Problem, Not Your Solution.*

One Final Takeaway

Before you go, I’d like to leave you with one final takeaway. We pay a lot of lip service to perseverance and grit, but perseverance and grit will get you only so far if you are trying to brute-force your solution.

Starting with a solution is like building a key without knowing what door it will open. You can try testing your key on lots of doors, or you can start with a door you want to open. When you fall in love with the problem versus your solution, you start building keys to doors that actually take you places.

Read Next:



The Science of How Customers Buy Anything